

Activation Sparsification in Transformers: Pressure, Thresholding, and Site Placement

Working title — methods and experimental setup — 5 September 2026

1 Introduction

Activation sparsity can reduce language-model inference cost when specialized kernels skip products and weight accesses associated with zero activations. Recent work shows that useful sparsity can be exposed in pretrained models through thresholding or encouraged during training, with measured acceleration in both cases (Liu et al., 2024; Cetin et al., 2026). These successes create a practical design problem: how should a researcher combine and place sparsification interventions to obtain more useful sparsity at an acceptable quality cost? Answering this question matters when choosing which components of a recipe to retain, where to invest additional training, and which operations warrant sparse-kernel development.

The available approaches act on different parts of this problem. Post-hoc thresholding exploits the tolerance of an existing representation; activation regularization changes what the model learns; and broader site placement exposes additional operations to sparsification. TEAL, ProSparse, and Q-Sparse establish the effectiveness of these approaches and investigate choices within them (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). These findings motivate examining the additional value of pressure as thresholding and placement change. An intervention that helps on its own may add little to another intervention, or impose a different quality cost when applied elsewhere. Without knowing these dependencies, an improved sparsity fraction gives limited guidance about why a recipe works or how to adapt it to a different model.

We study this problem through three separately controllable design choices: *pressure*, which encourages selected activations toward zero; *thresholding*, which sets values to exactly zero according to a gate; and *site placement*, which specifies where each acts. *The interventions are separately controllable, but their effects need not be separable.* Our central question is how their individual effects and interactions determine the quality–sparsity trade-off. In particular, does adding pressure become more or less useful as thresholding changes, and how does that response depend on where pressure and gates are applied?

Explaining that response requires connecting learned behavior to the computation it affects. A reduction in activation magnitude can leave the number of exact zeros unchanged; additional zeros can also be concentrated at sites that affect only a small share of the forward pass. Conversely, extending the gate set can expose additional operations to sparsification while changing the representation the model must learn. The empirical issue is how learning responds to these choices: when pressure produces additional zeros, how much quality that change costs, and which operations account for the result. Distinguishing these outcomes helps diagnose whether limited gains reflect the response to pressure, the computational reach of the chosen sites, or the cost of expanding the intervention.

To make that diagnosis model-wide, we pair observed zero-product counts with an architecture-dependent reach ceiling. $\mathcal{S}_{\text{model}}$ measures the fraction of counted forward matrix-multiplication products with a zero activation operand; $\mathcal{S}_{\text{model}}^{\text{max}}$ describes the fraction reachable if the selected sites were entirely zero. We refer to the former as *model-wide sparsity*, distinguishing it from a local activation zero fraction. This accounting makes the location of sparsity consequential: the same local zero rate can affect different amounts of computation as the architecture or workload changes. It helps distinguish a change in learned sparsity from a change in the work reachable through the chosen sites. Direct timing then tests whether an implementation can exploit the observed zeros on hardware.

We organize matched pretraining comparisons around a *sparsification intervention ladder* (Figure 1), which organizes gate and pressure alternatives across feed-forward (FFN) and attention sites. Pythia-family models provide a common architecture for examining these choices and their behavior

across sizes (Biderman et al., 2023). We use the ladder to trace individual and joint intervention effects from learned activations to the operations they affect. A transferable explanation must account for changes in those effects at another model size, where both learned responses and computational reach can differ. An account that meets this requirement can guide how sparsification recipes are adapted, including when to strengthen pressure, change a gate, or extend placement.

2 Related Work

2.1 Sparsification interventions

Post-hoc thresholding reveals sparsity that an already learned representation can tolerate. CATS thresholds small-magnitude activations within gated FFNs, with both training-free and fine-tuning settings (Lee et al., 2024). TEAL extends magnitude thresholding to attention and FFN projections and uses block-wise greedy allocation to account for their different sensitivities (Liu et al., 2024). Its placement and allocation studies show why a uniform sparsity target need not be equally appropriate throughout a model. These methods provide a reference for sparsity available without retraining, while motivating the question of how training can change that tolerance.

Training-time interventions allow the representation to adapt to sparse computation. ReLU Strikes Back studies restoring sparse activations in language models (Mirzadeh et al., 2023); ProSparse combines ReLU, progressive ℓ_1 pressure, and threshold shifting during continued training (Song et al., 2024). ProSparse’s schedule ablations show that comparable FFN sparsity can carry different quality costs. Sparsing Law examines how sparsity depends on regularization, data, and model size (Luo et al., 2025), while Cetin et al. (2026) demonstrate highly sparse FFNs induced by ℓ_1 regularization during pretraining. Together, these studies establish pressure as a way to reshape the quality–sparsity trade-off. A local FFN zero fraction alone leaves a further accounting question: how much of the complete model’s computation do those zeros affect?

Extending sparsification beyond FFNs makes the choice of sites explicit. Q-Sparse applies top- k selection to attention and FFN projection inputs, reports overall sparsity and activated-parameter budgets, and ablates both the sparsifying nonlinearity and the straight-through estimator (Wang et al., 2024). Spark Transformer combines sparse FFNs with sparse attention through token-position selection and examines the role of its low-cost predictor (You et al., 2025). Projection-input sparsity and attention-position sparsity affect different work; our intervention ladder also considers feature coordinates in the internal attention operands. Building on the existing component studies, we examine how the additional effect of activation pressure depends on thresholding and placement. Relating those effects to model-wide product counts and topology-dependent reach connects the quality cost of an intervention to the computation its zeros can affect.

2.2 Inference speedup

Converting activation sparsity into speedup requires an execution strategy suited to the workload. TEAL accelerates batch-one autoregressive decoding, where skipping weight accesses is valuable, and reports up to $1.8\times$ speedup at 50% sparsity under its projection-based accounting (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication: their TwELL format packs sparse activations as a projection produces them and supports fused FFN computation, reducing the overhead of materializing and consuming the sparse representation. They evaluate throughput for full-sequence forward passes, a different workload from token-by-token decoding. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). These results show why the location and execution of sparsity matter together. Skipped products and memory traffic must repay indexing, packing, and launch overheads, while operations that remain dense limit the gain of the complete model. A sparsity percentage alone does not specify this balance.

This dependence makes kernel adaptation part of evaluating whether a sparsification method transfers. With larger batches, different token masks can reduce the weight-loading savings available from sparsity, as Spark’s batching analysis illustrates (You et al., 2025). Block Q-Sparse introduces structure into the mask to accommodate batched execution (Wang et al., 2024); TwELL instead makes packing and fusion compatible with tiled computation (Cetin et al., 2026). Tensor shape, sparsity structure, and data layout therefore matter alongside the zero fraction. Iterative kernel generation with execution and profiling feedback offers an emerging tool for exploring such adaptations (Ouyang et al., 2025; Luo et al., 2026). Establishing a benefit still requires numerical correctness and complete-model timings that include selection and conversion costs on the target workload. These measurements test whether intervention-induced zero-product opportunities translate into reduced execution time after sparse-execution overheads are included.

3 Methodology

3.1 Sparsification interventions

A sparsification intervention specifies a gate-site set $\mathcal{G}$, gate families and thresholds at those sites, and a pressure method with target set $\mathcal{P}$. Gate and pressure sites are independently specified: the former determine where values are set to zero, and the latter where an auxiliary objective encourages small activations.

We use ReLU and two fixed-threshold gates: $G_{+, \kappa}(x) = x \mathbf{1}\{x \geq \kappa\}$ and $G_{\pm, \kappa}(x) = x \mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. One-sided thresholding removes negative values and small positive values; symmetric thresholding removes small magnitudes while preserving the signs and values of survivors. At $\kappa = 0$, their forward maps are ReLU and the identity, respectively. Appendix A specifies boundary and derivative conventions.

Pressure uses $\mathcal{L}_1$, the equally weighted mean of the mean absolute activations of each targeted site-layer tensor, taken after its gate when one is present. *Naive LI* optimizes $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$. *Orthogonal LI* (OL1) makes a task-only AdamW update, then adds a pressure correction preconditioned by the task’s second-moment estimates. When this correction opposes the adaptive task direction, OL1 projects out its conflicting component. A positive budget caps its norm relative to that task direction. This controls update geometry without guaranteeing preserved task loss; Appendix A.2 defines the procedure.

3.2 Sparsity and architectural reach

Local sparsity is the fraction of exactly zero entries at a named site. To weight zeros by the work they affect, we define *model-wide sparsity* as $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, reported as $100\mathcal{S}_{\text{model}}\%$. Here, N_0 counts scalar multiplications with at least one zero activation operand in the declared transformer-block matrix products. N_{model} counts all valid multiplications in those operations and the dense language-model head; the head receives no zero credit. Counts are pooled before division. The percentage refers to this declared multiplication workload, not all operations or elapsed time. We count only causally valid pairs in a full, uncached sequence; Appendix B gives the exact counters.

The *all-zero reach ceiling*, $\mathcal{S}_{\text{model}}^{\text{max}} = N_{\text{reach}}(\mathcal{G})/N_{\text{model}}$, counts the fraction of that workload whose zero operand is guaranteed by setting the selected gate sites entirely to zero, crediting each affected multiplication once. It depends on the architecture, site set, and sequence length, but not pressure or a checkpoint. The ceiling characterizes selected-site reach rather than sparsity achievable at acceptable quality: observed $\mathcal{S}_{\text{model}}$ also includes natural zeros outside that reach. Appendix B.2 defines the reach accounting.

4 Experimental study

We examine how pressure’s effects on quality and model-wide sparsity depend on thresholding and placement, and whether an explanation of those effects transfers across model sizes. We pre-train randomly initialized Pythia-14M, 70M, and 410M models (Biderman et al., 2023) on MiniPile (Kaddour, 2023). Their shared parallel attention and FFN architecture gives the sites a consistent interpretation across sizes. Detailed comparisons use 14M; the larger models cover selected recipes. We pair validation cross-entropy and $\mathcal{S}_{\text{model}}$ from the same checkpoint and forward configuration. Appendix C specifies the architecture and evaluation details.

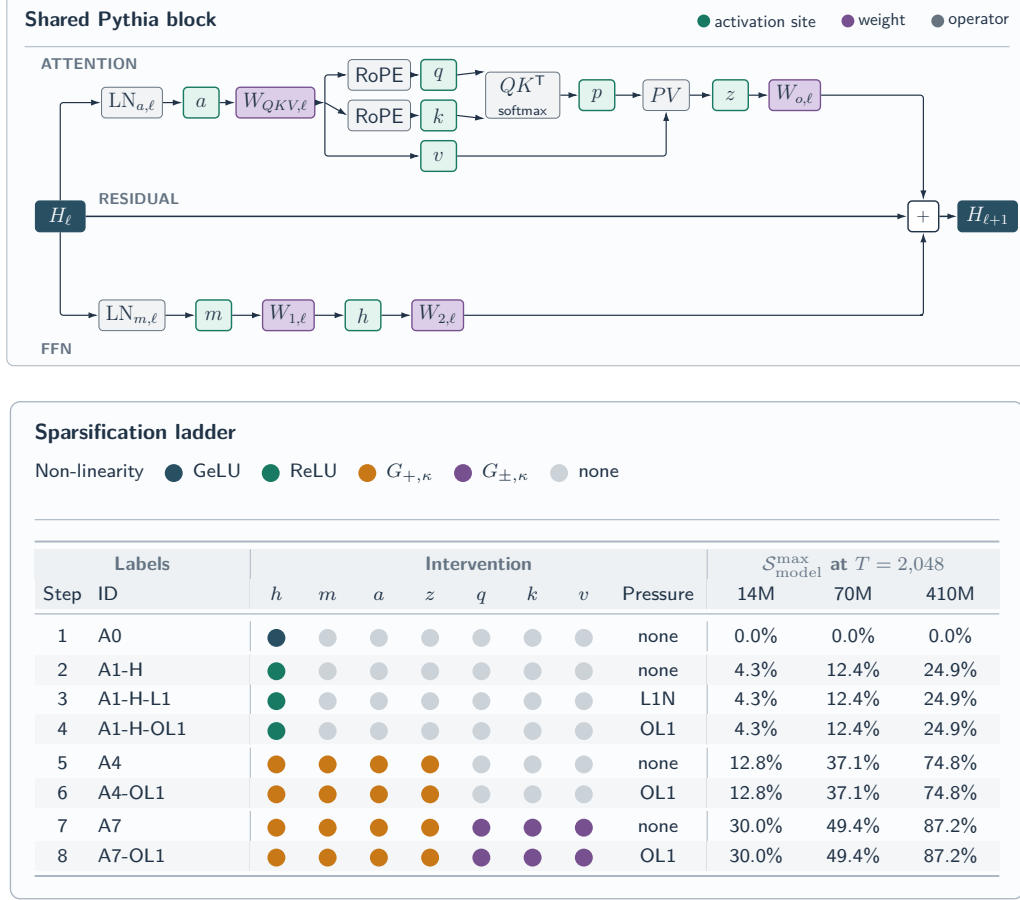


Figure 1: Shared Pythia block and sparsification intervention ladder. The map separates gate placement, gate family, and pressure; L1N denotes naive L1. The final columns show analytic selected-site reach 100 $\mathcal{S}_{\text{model}}^{\text{max}}$ % at $T = 2,048$, not observed sparsity or speedup. Pressure targets all gated sites in each pressure-bearing row. Rows define candidate recipes; their experimental coverage is reported with the results.

Figure 1 locates the normalized attention and FFN inputs (a, m), FFN hidden activation (h), concatenated attention context (z), and query, key, and value operands (q, k, v). Query and key gates follow rotary position encoding (RoPE); z precedes the attention-output projection. A0 retains the stock block. A1-H replaces GELU at h with ReLU; A4 applies one-sided thresholding at $\{a, m, h, z\}$; A7 adds symmetric gates at $\{q, k, v\}$. The L1/OL1 variants apply pressure to all gated sites in their row. These are separately pretrained conditions, not stages of a training curriculum. The ladder includes each recipe’s analytic reach, which changes with model architecture.

Within each size, matched contrasts share initialization, data order, and training budget. Adding pressure at fixed gates measures its additional quality and sparsity effects; repeating the contrast across thresholds tests how those effects depend on thresholding. Comparing A4 with A7 without pressure examines broader gate placement. A4-OL1 versus A7-OL1 changes both gate and pressure sets, so it compares complete recipes rather than isolating gate placement at a fixed pressure objective. At 70M and 410M, we evaluate A0, A1-H, A4-OL1, and A7-OL1 with the same training-token budget as at 14M. Comparisons use one initialization seed per size; thresholds are treatment levels, not statistical replicates. Across sizes, we examine each operation’s zero-product fraction and its share of the model workload to interpret changes in $\mathcal{S}_{\text{model}}$ alongside $\mathcal{S}_{\text{model}}^{\max}$.

References

- Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2304.01373>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. *arXiv preprint arXiv:2402.02998*, 2024. URL <https://arxiv.org/abs/2402.02998>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. *arXiv preprint arXiv:2404.08763*, 2024. URL <https://arxiv.org/abs/2404.08763>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Xiaojun Meng, Liquan Deng, Jiansheng Wei, Zhiyuan Liu, and Maosong Sun. Sparsing Law: Towards Large Language Models with Greater Activation Sparsity. *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://proceedings.mlr.press/v267/luo25i.html>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv preprint arXiv:2502.10517*, 2025. URL <https://arxiv.org/abs/2502.10517>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A Intervention definitions

A.1 Gate semantics

For finite $\kappa \geq 0$, both threshold gates retain equality and leave retained values unchanged. The comparison mask is detached: the derivative with respect to the input is one for retained entries and zero for rejected entries. Consequently, $G_{+,0}$ and ReLU have identical forward values but differ at zero, where their derivatives are one and zero, respectively. $G_{\pm,0}$ is exactly the identity in both values and gradients. The gates use neither a straight-through estimator nor a top- k selection rule. Unselected sites retain their original transformations.

A.2 Activation pressure and OL1

For the J targeted site-layer tensors A_j , containing n_j entries each, the pressure objective is $\mathcal{L}_1 = J^{-1} \sum_{j=1}^J n_j^{-1} \sum_{i=1}^{n_j} |A_{j,i}|$. The tensors contain the executed site activations, after a gate where present. Absolute values and means are evaluated in float32. Each tensor receives equal weight regardless of its size; adding pressure sites changes both the objective and the weight of existing targets. Task and pressure losses are averaged over the same accumulation microbatches. For naive L1, the accumulated gradient of $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$ is globally norm-clipped before AdamW (Loshchilov and Hutter, 2019).

For OL1, let g_t and g_1 be the accumulated task and unweighted pressure gradients. We first clip g_t by its global norm and perform AdamW using only that clipped task gradient. Let $\hat{m}_t$ and $\hat{v}_t$ be its newly updated, bias-corrected moments. On parameters having both gradients and initialized optimizer state, define the adaptive task and pressure vectors $u = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon_A)$ and $w = g_1 / (\sqrt{\hat{v}_t} + \epsilon_A)$. The pressure gradient is unclipped and has no first-moment buffer. Inner products and norms below pool all these eligible parameters. With $c = \langle u, w \rangle$, $q = \|u\|_2^2$, and numerical stabilizer $\epsilon > 0$, the conflict-conditioned projection is

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (1)$$

For pressure weight $\lambda \geq 0$ and required budget $b > 0$, define the norm ratio $r = \lambda \|\tilde{w}\|_2 / (\|u\|_2 + \epsilon)$. Set $s = \min\{1, b/(r + \epsilon)\}$ when $r > 0$, or $s = 1$ otherwise. After AdamW, each eligible parameter group p , with learning rate α_p , receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$. The experiments' clipping thresholds, b , and λ are condition parameters.

This procedure is related to gradient-conflict projection (Yu et al., 2020) and the use of smoothed primary directions (Hsieh et al., 2024), but defines a particular correction in AdamW-preconditioned coordinates. The stabilized projection is only approximately orthogonal under conflict. Its geometry and budget precede multiplication by group learning rates and exclude decoupled weight decay. They therefore do not guarantee task-loss descent or preservation.

B Sparsity accounting

B.1 Exact counters

Local exact sparsity at site s is $\mathcal{Z}_s = \sum_{\ell,e,i} \mathbf{1}\{A_{s,i}^{(\ell,e)} = 0\} / \sum_{\ell,e} |A_s^{(\ell,e)}|$, where ℓ indexes layers, e evaluation batches, i tensor entries, and $|A|$ the number of elements in tensor A . Near-zero mass replaces equality by $|A_{s,i}| \leq \epsilon$ for a stated tolerance; it does not contribute to exact-zero counts unless the value is zero. All aggregate fractions pool integer numerators and denominators across batches and layers before division.

The atomic unit of model-wide accounting is one scalar multiplication in a matrix product. If $X \in \mathbb{R}^{n \times p}$ feeds a linear projection with q outputs, the count is $N = npq$, with $N_0 = q \sum_{i,j} \mathbf{1}\{X_{ij} = 0\}$. Zero weights receive no credit. For the multihead attention graph specified in Appendix C.1, both operands are activations, so a product is credited once if either is zero. Within one block and evaluation batch, for query position t , key position $u \leq t$, and within-head coordinate j ,

$$N_{0,QK} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{Q_{bhtj} = 0 \vee K_{bhuj} = 0\}, \quad (2)$$

$$N_{0,PV} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{P_{bhtu} = 0 \vee V_{bhuj} = 0\}. \quad (3)$$

Here P is the causal softmax probability tensor; the model numerator sums these counts over blocks and evaluation batches. Future masked positions are excluded from both counts, while zeros on valid positions, including floating-point underflow, are included. Counts use the operands actually consumed by each matrix product; upstream masks do not determine those operands' zeros after intervening transformations. Earlier value positions participate in more causal pairs, so local value sparsity alone does not determine PV sparsity.

B.2 Reach accounting and normalization

For a specified graph and workload, let N_o count the valid scalar multiplications in block operation family o , pooled over layers and examples. Then $N_{\text{model}} = \sum_o N_o + N_{\text{head}}$, while N_0 sums the corresponding zero-operand counts only in block operations. The block-only view is $\mathcal{S}_{\text{block}} = N_0 / \sum_o N_o$. Both sparsities are fractions; percentages multiply them by 100. Embedding lookup, normalization, nonlinearities, softmax, RoPE, bias and residual additions, and data movement are outside this accounting.

To construct the reach numerator, count products guaranteed to have a zero activation operand as a structural consequence of setting the selected sites to zero, independently of checkpoint values or natural zeros elsewhere. Include exact zero-preserving propagation under the declared graph rules. Products reached through more than one site receive one credit. The resulting count is $N_{\text{reach}}(\mathcal{G})$. Division by the same N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$. Natural zeros outside this selected-site reach can contribute to observed $\mathcal{S}_{\text{model}}$, so the ceiling is not an upper bound on every observation. Its gap to an observation does not measure sparsity available without quality loss. Appendix C.2 specializes the denominator and reach rules to the Pythia graph used in the experiments.

C Experimental setup details

C.1 Pythia sites and gate placement

Pythia uses parallel attention and FFN branches with separate LayerNorms. For batch size B , sequence length T , residual width d , FFN width d_f , and H heads, let $d_h = d/H$. Table 1 identifies the actual operands recorded after any selected gate. The same site mapping is applied in every layer, with one common threshold per condition. The recipe and pressure assignments are defined in Section 4.

At an ungated h , the activation is GELU; a selected gate replaces GELU. At other ungated sites the map is the identity applied to that port's input. At zero threshold, A7's additional symmetric query/key/value gates are exact identities in both values and gradients. They therefore do not change A4's mathematical forward map in the absence of pressure. A7-OL1 still differs from A4-OL1 in its pressure targets and their equal-tensor normalization.

Query and key counts use actual post-RoPE operands: rotation can repopulate an individual zero coordinate. Likewise, the output projection uses actual post-gate context z , rather than inferring its zeros from the value mask.

Table 1: Intervention sites and the matrix products that consume them directly.

Site	Shape	Consuming operation	Placement
a	$B \times T \times d$	QKV projection	After attention LayerNorm
m	$B \times T \times d$	FFN up projection	After FFN LayerNorm
h	$B \times T \times d_f$	FFN down projection	Nonlinearity output
q, k	$B \times H \times T \times d_h$	Attention scores	After partial RoPE
v	$B \times H \times T \times d_h$	Probability-value	After the QKV split
z	$B \times T \times d$	Attention output	Concatenated context, before output projection

C.2 Architecture-specific counts and reach

The operation inventory comprises QKV projection, attention scores (QK), probability-value products (PV), attention-output projection, and the two FFN projections. For one uncached, unpadded sequence, each block contributes $3Td^2$ products from QKV projection, Td^2 from the attention-output projection, Tdd_f from each FFN projection, and $dT(T+1)/2$ from each of QK and PV . Thus, for L blocks and vocabulary size V_{vocab} ,

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (4)$$

Over B sequences, $N_{\text{model}} = BC_{\text{model}}$; the dense head contributes only to the denominator, as defined in Section 3.2.

Let $\mathcal{O}(\mathcal{G})$ be the union of operations reachable when the selected sites are all zero: a reaches QKV; m reaches FFN up; h reaches FFN down; either q or k reaches QK ; v reaches PV and attention output; and z reaches attention output. Each operation is counted once. The v rule includes the exact closure $V=0 \Rightarrow PV=0$. No further propagation is assumed: QKV biases can survive $a=0$, and zero scores do not imply zero softmax probabilities. For the same site set in every layer, $N_{\text{reach}} = BL \sum_{o \in \mathcal{O}(\mathcal{G})} C_o$, where C_o is that operation’s per-sequence count. Division by N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$.

With Pythia’s $d_f = 4d$, the per-block reachable counts for A0, A1-H, A4, and A7 are 0, $4Td^2$, $12Td^2$, and C_{block} , respectively. The ladder evaluates these formulas at $T = 2,048$, $V_{\text{vocab}} = 50,304$, and the pinned (L, d) configurations (6, 128), (6, 512), and (24, 1,024).¹ No checkpoint enters this calculation. In particular, A0 has zero selected-site reach but may have positive observed sparsity from natural zeros. The ceiling is consequently not an upper bound on every observed $\mathcal{S}_{\text{model}}$, nor does its gap to an observation measure sparsity available without quality loss.

C.3 Evaluation scope

Quality and logical counts are paired in the same eager-attention evaluation of a checkpoint, with its declared gates and evaluation precision. The complete MiniPile validation split contains 500 documents. Concatenation and packing produce 338 complete blocks of 2,048 tokens (692,224 input tokens); the final 1,444-token tail is excluded. Each block contributes 2,047 next-token targets to cross-entropy, while product accounting includes all 2,048 input positions. Loss is averaged over the equally sized blocks, and sparsity is count-pooled. This workload uses no key-value cache. Condition-specific training budgets, optimizer settings, seeds, and checkpoint selection accompany their results; they are not implied by the ladder’s list of candidate conditions.

¹Architecture configurations: EleutherAI’s Pythia-14M-deduped, Pythia-70M-deduped, and Pythia-410M-deduped. Links identify immutable configuration revisions; released weights are not used.